WEB MOBILE · WEBLAB · ETAPA 2

PetConecta
Arquitetura, rotas e API

Como o projeto em Next.js divide o trabalho entre servidor e cliente, quais rotas existem e como os dados em JSON chegam às páginas pela API usando `fetch`.

STACK
Next.js 16 · React 19 · App Router

GRUPO
Gabriel Pissoli Padrão · Luana Medeiros · Mariana Dantas

DATA
Novembro de 2026

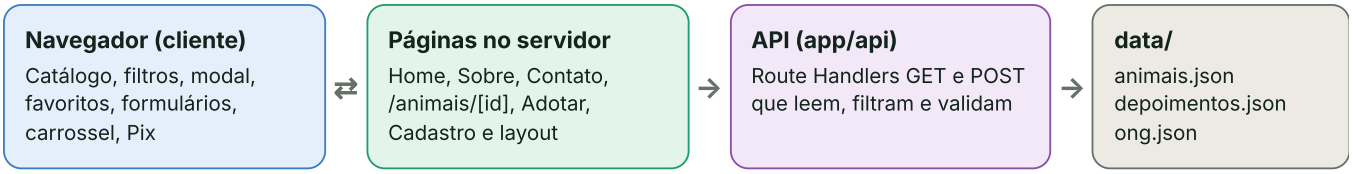
01 Servidor × Cliente
O que roda em cada lado e por quê

02 Rotas e mapeamentos
Páginas, endpoints da API e arquivos JSON

03 JSON via API com fetch
Como o projeto garante essa regra

1 Servidor e cliente

No Next.js, a diferença entre os dois lados é **onde o código roda**. Componentes sem a diretiva `"use client"` rodam no servidor. Os que têm a diretiva rodam no navegador.



O cliente também chama a API diretamente: o catálogo busca `/api/animais` e os formulários enviam POST.

SERVIDOR Guarda e entrega os dados

- **API** (`app/api/.../route.js`): é a única parte que lê os JSON. Filtra os dados e responde em JSON. Nas rotas POST, valida com `lib/validacao.js` e responde **201** (com protocolo) ou **400** (campo faltando ou email inválido).
- **Páginas sem interação** (Server Components): Home, Sobre, Contato, `/animais/[id]`, Adotar e Cadastro. O HTML é montado antes de chegar ao navegador. As quatro primeiras buscam dados com `apiGet()`.
- **Layout** (`app/layout.jsx`): estrutura comum com Header, `<main>` e Footer.

CLIENTE Cuida da interação

- `CatalogoAnimais` e `FiltroAnimais`: buscam de novo na API a cada filtro, sem recarregar.
- `AnimalCard` e `AnimalModal`: abrem a ficha e marcam favoritos.
- `useFavoritos`: salva no `localStorage`, que só existe no navegador.
- `FormAdocao`, `FormCadastro`, `FormContato` e `useEnvioForm`: enviam e mostram a resposta.
- `CarrosselDepoimentos`, `CalculadoraDoacao` e `CopiarPix` (área de transferência).
- `Header`: menu mobile com estado do React.

Exemplo dos dois lados juntos: a Home

A página busca `/api/ong` e `/api/depoimentos` no servidor e entrega os dados prontos, por props, para o Carrossel, a Calculadora e o CopiarPix. Esses três componentes rodam no navegador, mas não chamam a API sozinhos.

2 Rotas e mapeamentos

O App Router cria uma rota para cada pasta com `page.jsx`. As URLs ficam sem `.html`, e cada página busca o dado de uma rota da API.

Páginas: antes, agora e origem do dado

HTML antigo	Rota Next	Arquivo	Onde roda	Dado
`ong.html`	**/**	`app/page.jsx`	servidor	`GET` /api/ong + /api/depoimentos
`sobre.html`	**/sobre**	`app/sobre/page.jsx`	servidor	`GET` /api/ong (FAQ)
`caes.html`	**/caes**	`app/caes/page.jsx`	cliente	`GET` /api/animais?especie=cao + filtros
`gatos.html`	**/gatos**	`app/gatos/page.jsx`	cliente	`GET` /api/animais?especie=gato + filtros
(nova)	**/animais/[id]**	`app/animais/[id]/page.jsx`	servidor	`GET` /api/animais/[id] (404 se não existir)
`adotar.html`	**/adotar**	`app/adotar/page.jsx`	servidor + form	`POST` /api/adocao
`cadastro.html`	**/cadastro**	`app/cadastro/page.jsx`	form no cliente	`POST` /api/cadastro
`contato.html`	**/contato**	`app/contato/page.jsx`	servidor + form	`GET` /api/ong + `POST` /api/contato

API: endpoint, arquivo lido e função

Método	Rota	Lê	O que faz
`GET`	`/api/animais`	animais.json	Lista os animais, com filtros opcionais: `especie`, `sexo`, `idade`, `porte`, `energia`, `raca`, `ambiente`
`GET`	`/api/animais/[id]`	animais.json	Devolve um animal, ou `{ erro }` com status 404
`GET`	`/api/depoimentos`	depoimentos.json	Devolve todos os depoimentos
`GET`	`/api/ong`	ong.json	Devolve impacto, Pix, doação, contato e FAQ
`POST`	`/api/adocao`	—	Exige `assunto`, `animal` e `email`
`POST`	`/api/cadastro`	—	Exige os 8 campos do tutor e do animal
`POST`	`/api/contato`	—	Exige `assunto`, `telefone` e `mensagem`

Ligação entre páginas

O botão "Quero Adotar" do modal e da página do animal leva para `/adotar?animal=Nome`. Lá, o `FormAdocao` já abre com o nome do animal preenchido.

3 O JSON é consumido só pela API, com fetch

Sim, é obrigatório no projeto, e a estrutura foi montada para garantir isso de três formas.

1. **`data/` fica fora de `public/`.** Os JSON não têm endereço público: ninguém consegue abrir `/animais.json` no navegador.

2. **O `import` do JSON só existe em `app/api/`.** Nenhuma página ou componente importa os arquivos. Na rota, o import é o próprio servidor lendo seu "banco de dados".

3. **Toda página chega aos dados por `fetch`**, em exatamente três lugares do código, mostrados abaixo.

components/CatalogoAnimais.jsx — navegador · GET

```
const controle = new AbortController();
fetch(`/api/animais?${params}`, { signal: controle.signal })
  .then((res) => res.json())
  .then((lista) => setAnimais(lista));
```

URL relativa. O `AbortController` cancela a busca anterior quando o filtro muda.

lib/api.js · apiGet() — servidor · GET

```
const base = process.env.NEXT_PUBLIC_SITE_URL ?? `${protocolo}://${host}`;
const res = await fetch(`${base}${caminho}`, { cache: "no-store" });
return await res.json();
```

No servidor, a URL precisa ser absoluta. O `no-store` impede que o `next build` chame a API antes de o servidor existir.

lib/useEnvioForm.js — navegador · POST

```
const res = await fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(dados),
});
const corpo = await res.json(); // { mensagem, protocolo } ou { erro }
```

Usado pelos três formulários: adoção, cadastro e contato.

Resumo para a apresentação

O dado só sai do arquivo JSON passando pela API, e as páginas só chegam à API com `fetch`. A resposta é sempre lida com `res.json()`.

Ajuste antes da entrega

O `ESPECIFICACAO.md` e o `README.md` citam TypeScript (`page.tsx`, `route.ts`, `lib/types.ts`), mas o código foi escrito em JavaScript (`.jsx`/`.js`) e não existe `types.ts`. Vale alinhar os documentos com o código.